

Indications pour les questions de programmation

Question 1 Pas de grosse difficulté à cette question. Il faut penser à initialiser le tableau des états après avec des `false` avant de le remplir.

Pour faire les tests, on peut se créer une fonction qui crée un tableau de `false` (ou utiliser `calloc` si on sait s'en servir).

Question 2 Un classique sur les automates : on lit une transition pour chaque lettre en partant de l'ensemble ne contenant que 0. On pense à libérer la mémoire.

Question 3 Cette question est assez difficile à implémenter efficacement, pour pouvoir traiter des mots de taille 200. Il faut ici procéder par programmation dynamique.

Si on note $w = a_0 \dots a_{n-1}$, et $w_i = a_i a_{i+1} \dots a_{n-1}$, alors on peut calculer récursivement, pour $q \in Q$, le nombre de chemins étiquetés par w_i de q à l'unique état final q_{N-1} . Notons $C(q, i)$ cette quantité.

- On remarque que $C(n, q) = \begin{cases} 1 & \text{si } q = q_{N-1} \\ 0 & \text{sinon} \end{cases}$;
- pour $q \in Q$ et $i \in \llbracket 0, n-1 \rrbracket$, $C(i, q) = \sum_{p \in \Delta(q, u_i)} C(i+1, p)$.

Grâce à cette formule, on peut faire un calcul de tous les $C(i, q)$, pour $i \in \llbracket 0, n \rrbracket$ et $q \in Q$, de manière récursive mémoïsée.

La valeur cherchée est $C(0, q_0)$.

Par ailleurs, pour les tests, il faut créer les chaînes des réponses b) et c) par des boucles. Inutile d'allouer ces chaînes sur le tas, des tableaux statiques suffisent.

Question 4 Cela revient à parcourir tous les mots à deux lettres de taille inférieure à la longueur maximale. On peut pour cela utiliser une classique fonction d'incrémement, par exemple dans un tableau rempli de `'\0'` initialement pour que les chaînes soient parcourues par taille croissante. On utilise la fonction précédente pour chaque chaîne ainsi testée.

Question 5 Sachant qu'on cherche les distances, il faut ici implémenter un parcours en largeur (on peut s'en douter vu que l'énoncé nous rappelle l'utilisation des files). L'utilisation des types `option` est un peu pénible, mais obligatoire étant donné l'utilisation de la fonction `hash_ioa`.

Question 6 On peut à nouveau s'en sortir avec un BFS, mais il faut l'adapter. On peut travailler en deux temps à chaque itération : tant que la file est non vide, on lit des `a`, et on ajoute tous les nouveaux états atteints à un nouvel ensemble. Une fois que la file est vide, on lit un `b` depuis chacun des états ainsi atteints, en augmentant leur distance de 1, et on ajoute à la file tous les nouveaux états atteints en lisant ce `b` (et on recommence).

Question 7 Avec la question 5, on peut obtenir les états accessibles. Si on transpose l'automate, on obtient les états coaccessibles. En faisant l'intersection, on a les états utiles. Il est ici plus pratique de passer par l'automate transposé pour se contenter de sommer les distances ainsi obtenues pour chaque état.

Question 8 Cette question est assez difficile si on ne voit pas l'automate à construire décrit dans l'indication. Il s'agit ici de construire un automate produit de l'automate A avec lui-même ! Dans cet automate produit $A \times A$, si on a (q_0, q_0) comme état initial et (q_{N-1}, q_{N-1}) comme état final, alors A est ambigu si et seulement s'il existe un état (p, q) utile dans $A \times A$, avec $p \neq q$. En effet, cela signifie qu'il existe un mot w dont une lecture passe par p et une autre lecture passe par q .

Question 9 On peut calculer les composantes fortement connexes (avec Kosaraju, directement sur l'au-

tomate, ou en le convertissant en graphe). Dès lors, il existe un $a\Sigma^*$ -cycle s'il existe un état p dans la même CFC qu'un état q tel que $p \xrightarrow{a} q$.

Question 10 On peut à nouveau utiliser les CFC : s'il y a une transition étiquetée par b entre deux états utiles d'une même CFC, alors le nombre maximal de b dans un mot accepté est infini. Sinon, on peut calculer le nombre maximal de b en ne comptant que les b entre deux composantes fortement connexes. L'idée est similaire à au calcul de la longueur maximale d'un chemin dans le méta-graphe (qui est sans cycle).

Question 11 Après avoir créé l'automate A' , on peut créer un automate particulier A_{deg} , ayant les mêmes états que A , et dont les transitions sont données par :

- $p \xrightarrow{a} q$ s'il existe un chemin de p à q dans A ;
- $p \xrightarrow{b} q$ s'il existe un $a\Sigma^*$ -cycle depuis (p, q, q) dans A' .

Dès lors, le degré de A est le maximum du nombre de b dans un mot accepté par A_{deg} .